School of Science

EDITH COWAN UNIVERSITY

Programming Principles (CSP1150)

Assignment 2: Individual programming project ("Pokédex" Program)
Assignment Marks: Marked out of 35, worth 35% of unit
Due Date: 2 November 2026, 9:00AM

A Message Regarding Academic Integrity

This unit has experienced academic misconduct including seeking/paying for solutions, working with/copying from other students, and using AI tools. This has resulted in students receiving 0 marks for the assessment or entire unit, suspensions, and even expulsion from the university.

If you start your assignments in a timely fashion, engage with the unit content, and seek help and feedback from unit staff and ECU learning support staff, these issues will not apply to you.

Background Information

This assignment tests your understanding of and ability to apply the programming concepts we have covered throughout the unit. The concepts covered in the second half of the unit build upon the fundamentals covered in the first half of the unit.

You are required to **use Git for version control** of your work and **submit your work as a Git bundle**. See the "**Using Git for Assessments**" document provided with this brief for details.

Contact your tutor if you have questions or do not understand any part of the assignment, or if you would like help or feedback on your work. You should do this *at least once* prior to submission.

Assignment Overview



This assignment involves interacting with an online Pokémon information service to manage and display information about various Pokémon.

You are required to design and implement two related programs:

- "**admin.py**", a CLI program that allows the user to manage a collection of Pokémon, getting details from an online service and storing them in a text file.
- "**pokedex.py**", a GUI program that allows the user to view details of the Pokémon stored in the text file, including an image, and navigate between them.

The following pages describe the requirements of both programs in detail.

Starter files for both programs are provided along with this assignment brief to help you get started and to facilitate an appropriate program structure. *Please use the starter files.*

Tip: Read the entire brief carefully, and refer back to it as you work on the project.
Visit the Discussions on the unit site regularly for extra information, tips and examples.

Pseudocode

As emphasised by the case study of Module 5, it is best to *design* a solution before starting to write code. You are required to *write and submit pseudocode of your program design* for *"admin.py"*, but not "pokedex.py" (pseudocode is not well suited to depicting the design of event-driven GUI programs). Tutors are happy to provide help and feedback throughout the semester, but they will expect you to be able to show your pseudocode and explain the design of your code.

Pseudocode is most beneficial if you attempt it *before* trying to code your program – even just a rough draft of the overall structure or a single section of the program is useful. Revise and expand it as you work on the code. This back-and-forth cycle of designing and coding is normal and expected, particularly for new programmers, and deepens your understanding. *The requirements on the following pages should give you a good idea of the structure of the program, allowing you to make a start on designing your solution in pseudocode.*

See Reading 3.3 and the discussion board for further advice and tips regarding writing pseudocode.

Your pseudocode document should contain a section for the *main program*, followed by a section of *pseudocode for the function(s)* defined in your program. The pseudocode for each function should clearly detail the parameters it receives, what it does, and what it returns to the program.

Think of the pseudocode of the main program as the *content* of a book, and the pseudocode of functions as its *appendices*: You can read and understand a book without necessarily reading the appendices, however they are there for further reference if needed without getting in the way.

The function(s) required in "admin.py" are detailed later in the assignment brief.

Working with PokéAPI

This project involves using PokéAPI – an online service providing information about Pokémon. Reading 7.4 introduces making API requests and using the responses.



PokéAPI is free and does not require an API key to use, but does requires requests to include a "User-Agent" header. To account for this, **use the following approach to make requests**:

```
request  = urllib.request.Request(URL, headers={'User-Agent': STUDENT_NUM})
response = json.load(urllib.request.urlopen(request))
```

This first creates a "request object" specifying a URL to request and including a User-Agent header, then uses it to make the request. Replace **`URL`** with the URL to request, and **`STUDENT_NUM`** with your student number. *Further guidance will be posted to the discussion board!*

Remember that your project *cannot* use any external modules.

Responses are received in JSON format, covered in Lecture 6 and Reading 7.1. Since they contain *a lot* of information and only a small amount of it is relevant to the project, **abridged examples of the response structure will be posted to the discussion board** for your convenience.

Take the time to develop an understanding of making requests and making use of the responses received, e.g. by experimenting with it in separate little programs: *It will be very helpful to you!*

Overview of "admin.py"

"admin.py" is a program with a Command-Line Interface (CLI) like the programs we have created throughout most of the unit. It can be implemented in under 225 lines of code (although optional additions may result in a longer program). *This is not a limit or a goal, but you should ask your tutor for advice if your program significantly exceeds it.* Everything you need to know for this program is covered in the first 7 modules of the unit.

This program allows the user to manage a collection of Pokémon, each consisting of a name, number, category, height, weight, types, and which Pokémon it evolves from (if applicable). They can add new Pokémon, and list, search, view and delete existing ones.

The data is stored in a text file named "pokemon.txt". Use the "`json`" module to write data to a text file in JSON format and to read the JSON data from the file into Python. *See Lecture 6 and Reading 7.1 for details about this.*

Below is an example of the data in JSON format (*you must use the same structure, types and keys)*:

JSON

```
[
    {
        "name": "Bulbasaur",
        "number": 1,
        "category": "Seed Pok\u00e9mon",
        "height": 0.7,
        "weight": 6.9,
        "types": ["grass", "poison"]
    },
    {
        "name": "Ivysaur",
        "number": 2,
        "category": "Seed Pok\u00e9mon",
        "height": 1.0,
        "weight": 13.0,
        "types": ["grass", "poison"],
        "evolves_from": "Bulbasaur"
    }
]
```

This example contains details of two Pokémon, stored in a *list*. Each Pokémon's details are stored in a *dictionary* containing keys of "name", "number", "category", "height", "weight" and "types". A key of "evolves_from" is also included if applicable.

The "name", "category" and "evolves_from" keys contain *strings*, "number" contains an *integer*, "height" and "weight" contain *floats* (in metres and kilograms), and "types" contains a *list of strings*.

("\u00e9" is just how "é" is represented)



If this data was stored a Python variable named `data`, then `data[0]` would refer to the entire dictionary of Bulbasaur's details, and `data[0]['weight']` would refer to 6.9.

Understanding the structure of the data and how to interact with it is *crucial* throughout the project. In particular, you will need to understand how to loop through the items of a list and how to refer to specific items in a dictionary. Revise Modules 3 and 7 if you are unsure about how to interact with lists and dictionaries, and **see the discussion board on the unit site** for further help.

Note: Remember to *create a new Git repository* for the project before you start working on it, and commit to it whenever you make significant changes or progress!

Output Example of "admin.py"

To help you visualise the program, here is a screenshot of it being used:

(This example assumes the existence of "pokemon.txt" containing the JSON data on Page 3)

```
Welcome to the Pokédex Administration Program.

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> l
Your Pokédex contains:
 #0001: Bulbasaur
 #0002: Ivysaur

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> a
Enter a Pokémon name: squirtle
Squirtle added to Pokédex.

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> a
Enter a Pokémon name: charmander
Charmander added to Pokédex.

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> l
Your Pokédex contains:
 #0001: Bulbasaur
 #0002: Ivysaur
 #0004: Charmander
 #0007: Squirtle

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> s
Enter search term: saur
Search results:
 #0001: Bulbasaur
 #0002: Ivysaur

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> v
Pokémon number to view: 2
Ivysaur [#0002]
 Seed Pokémon
 Type: Grass, Poison
 Size: 1.0m, 13.0kg
 Evolves from Bulbasaur

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> d
Pokémon number to delete: 2
Ivysaur deleted from Pokédex.

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> l
Your Pokédex contains:
 #0001: Bulbasaur
 #0004: Charmander
 #0007: Squirtle

Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit.
> q
Goodbye!  Made by Greg (12345678).
```

Whenever you display Pokémon numbers, zero-pad them to 4 digits and include a # at the start.

Store/list Pokémon in numerical order, regardless of the order in which they were added.

Display Pokémon details exactly as pictured here, Including the layout/presentation of types and size.

Terminology note: A "Pokédex" is a handheld device used to record/view Pokémon details.



Requirements of "admin.py"

The numbered points describe *core requirements*, and the bullet points are *additional details and hints* about them. Ask your tutor if you do not understand anything or want further information.

*Remember to **commit to your Git repository** whenever you make significant changes or progress!*

1. The first thing the program should do is try to open a file named "pokemon.txt" in read mode, then load the data from the file into a variable named `data` and then close the file.
   - *The data in the file should be in JSON format, so you will need to use the "`load()`" function from the "`json`" module to read the data into your program. See Page 3 for details of the structure.*
   - *If any exceptions occur (e.g. due to the file not existing, or it not containing valid JSON data), then simply set the `data` variable to be an empty list. This will occur the first time the program is run, since the data file will not exist yet. This ensures that you are always left with a list named `data`.*
   - *This is the only time that the program should need to read anything from the file. From this point the program uses the `data` variable, which is written to the file whenever anything is added or deleted.*

2. The program should then show a welcome message and enter an endless loop which starts by showing a list of options: "Choose [a]dd, [l]ist, [s]earch, [v]iew, [d]elete or [q]uit." and then prompts the user to enter their choice. Once a choice has been entered, use an "`if/elif`" statement to handle each of the different choices (detailed in the following requirements).
   - *This requirement has been completed for you in the starter file.*

The "add" section is the **hardest part of the project**, and involves things not covered in depth in the unit. Support will be provided via the discussion board, email, and in person/live streams.

3. If the user enters "`a`" (add), prompt them to enter the name of a Pokémon. Remove any full stops, apostrophes and colons from the input and replace any spaces with hyphens to ensure that the name is suitably formatted, then request the Pokémon's details from PokéAPI:

   First, concatenate the name to the end of **http://pokeapi.co/api/v2/pokemon-species/** and request it. If the name does not match a Pokémon, a urllib.error.HTTPError exception will occur: Show a suitable message and return to the menu. If the name is valid, the response includes the Pokémon's number, name, category, and which Pokémon it evolves from. If the `data` list already contains this Pokémon, show a suitable message and return to the menu.
   - *Use "`input_something()`" (detailed below) when asking the user for a Pokémon name, to ensure that they are re-prompted until they enter something other than whitespace.*
   - *The number we want is the "national Pokédex number", which is in the "entry_number" key of the first dictionary in the "pokedex_numbers" list in the response.*
   - *The name we want is in the "name" key of a dictionary in the "names" list in the response. Use "`extract_english()`" (detailed below) to get the English one.*
   - *The category we want is in the "genus" key of a dictionary in the "genera" list in the response. Use "`extract_english()`" (detailed below) to get the English one.*
   - *If the Pokémon evolves from another Pokémon, it will be detailed in the "evolves_from_species" key. An additional request will be needed to get the proper English name of the earlier Pokémon.*

Next, concatenate the Pokémon's *number* to the end of **http://pokeapi.co/api/v2/pokemon/** and request it. The response includes the Pokémon's height, weight, types, and URLs of images of it. Request the image at the "front_default" key of the "sprites" dictionary and save it into an "images" folder (create it if it does not exist), using the Pokémon's number in the filename.
e.g. The image of Bulbasaur would be named "1.png".

- *The "height" and "weight" keys contain values in decimetres and hectograms (why?!) – divide the values by 10 to convert them to metres and kilograms.*
- *See the discussion board for guidance regarding creating folders and requesting/saving image files!*

Create a dictionary of the Pokémon's details with the structure shown on Page 3 and add it to the `data` list, ensuring that the list is ordered by Pokémon number. Then, write the entire `data` list to the text file in JSON format to save the data.

- *The easiest way to order the `data` list by Pokémon number is to append the new dictionary to the end of it then use "`data.sort(key=lambda p: p['number'])`" to order it correctly.*
- *Once the `data` list contains the new Pokémon and is correctly ordered, call "`save_data()`" (detailed below) to write the data to the text file in JSON format.*

You can work on other parts of the project before completing the "add" section by copying the JSON example from Page 2 into a file named "pokemon.txt" for the programs to use.

4. If the user enters "`l`" (list), display the number and name of all Pokémon in the `data` list, as pictured on Page 4.
   - *If the `data` list is empty, show a suitable message instead.*
   - *Use a "`for`" loop to iterate through the items in the `data` list. Remember: Each item is a dictionary.*

5. If the user enters "`s`" (search), prompt them for a search term and then list any Pokémon whose name or category contains the search term. Display results in the same way as when listing.
   - *If the `data` list is empty, show a suitable message instead of asking for a search term.*
   - *Use "`input_something()`" (detailed below) when prompting for a search term, to ensure that the user is re-prompted until they enter something other than whitespace.*
   - *The code to search will be like the code used to list, but with an "`if`" statement in the loop body to only include Pokémon that contain the search term (use the "`in`" operator – see Lecture 3).*
   - *Ensure that the searching is not case-sensitive, e.g. "pikachu" should find "Pikachu".*
   - *If the search term is not found in any of the Pokémon, show a suitable message.*

6. If the user enters "`v`" (view), prompt them for a Pokémon number and then show the selected Pokémon's details as pictured on Page 4.
   - *If the `data` list is empty, show a suitable message instead of asking for a number.*
   - *Only show the "Evolves from..." line if the "evolves_from" key exists in the Pokémon's dictionary.*
   - *Use "`choose_pokemon()`" (detailed below) when asking the user to enter a Pokémon number.*



7. If the user enters "`d`" (delete), prompt them for a Pokémon number and then delete the selected dictionary from the `data` list, delete the Pokémon's image from the "images" folder, then show a suitable message.
   - *If the `data` list is empty, show a suitable message instead of asking for a number.*
   - *Use "`choose_pokemon()`" (detailed below) when asking the user to enter a Pokémon number.*
   - *Since "`choose_pokemon()`" returns the selected dictionary, use the "`remove()`" list method to delete it from the `data` list.*
   - *Use "`os.remove()`" to delete the image file.*
   - *Once the dictionary has been deleted from the `data` list, call "`save_data()`" (detailed below) to write the data to the text file in JSON format.*

8. If the user enters "`q`" (quit), show a goodbye message that includes *your* name and student number, and `break` out of the loop to end the program.

9. If the user enters anything else, show a suitable message (the user will then be re-prompted for a choice on the next iteration of the loop).

This concludes the core requirements of "admin.py". The following pages detail the functions mentioned above and optional additions and enhancements that can be added to the program.

Remember that you are required to submit pseudocode for your design of "admin.py". Use the starter file and requirements above as a guide to structure the design of your program.

Tip: The requests we are sending to PokéAPI can end in either a Pokémon name *or their number*. Hence, when adding a Pokémon you can enter a number, e.g. "25", instead of a name.

This can make it quicker/easier to test your code. There are about 1000 different Pokémon!

Functions in "admin.py"

The requirements mentioned 4 functions – "`input_something()`", "`extract_english()`", "`choose_pokemon()`", and "`save_data()`". Your program must define and use them.

1. The "`input_something()`" function takes 1 parameter named `prompt` (the message to show before waiting for input). It repeatedly asks the user (using the `prompt`) for input until they enter a value consisting of at least 1 non-whitespace character (i.e. the input cannot be nothing or entirely spaces, tabs, etc.). It then returns the value as a string.
   - *Use the "`strip()`" string method on a string to remove whitespace from the start and end. If a string consists entirely of whitespace, it will be empty once you strip the whitespace away.*

2. The "`extract_english()`" function takes 2 parameters named `entries` (a list of dictionaries) and `key` (a string). It finds and returns the value of the specified `key` of the first dictionary in the `entries` list that is in English. An English entry can be identified by a value of "en" in the "name" key of the "language" dictionary in each entry's dictionary.
   - *The `entries` that you pass to this function will be the lists of names and categories (found in the "names" and "genera" keys) located within the response of "pokemon-species" requests.*
   - *See the discussion board to better understand the structure of the data you're working with!*

```
name = extract_english(species_data['names'], 'name')
```

3. The "`choose_pokemon()`" function takes 2 parameters named `prompt` (the message to show before waiting for input), and `data` (the main program's list of Pokémon). It repeatedly asks the user (using the `prompt`) for input until they enter an integer matching the number of one of the Pokémon in the `data` list. It then returns the selected Pokémon's dictionary.

4. The "`save_data()`" function takes 1 parameter named `data`. The function should open "pokemon.txt" in write mode, then write the `data` to the file in JSON format and close the file. This function does not return anything.
   - *This is the only part of the program that should be writing to the file, and it always overwrites the entire content of the file with the entirety of the current data.*
   - *See Lecture 6 and Reading 7.1 for coverage of the "`json`" module. You can specify an additional `indent` parameter in the "`dump()`" function to format the JSON data nicely in the text file.*

Ensure that the functions do exactly what is specified above and nothing more – it is important to adhere to the stated specifications of a function, and deviating from them will impact your marks.

In particular, the "`prompt`" parameter of "`input_something()`" and "`choose_pokemon()`" is for *the text that you want to show as a prompt*. Here are some examples of using the functions:

```
search = input_something('Enter a search term: ')
Enter a search term: saur

pokemon = choose_pokemon('Pokémon number to view: ', data)
Pokémon number to view: 2
```

You are welcome to define and use additional functions if you feel they improve your program, but consider the characteristics and ideals of functions as outlined in Lecture 4.

Optional Additions and Enhancements for "admin.py"

Below are suggestions for minor additions and enhancements that you can implement to further test and demonstrate your programming ability.

They are *not required,* and you can earn full marks without implementing them.

1. Make "`choose_pokemon()`" return False if the user enters 0, allowing for a way to "cancel". Incorporate this into the "view" and "delete" sections of the program.

2. When adding a Pokémon, include the first English "flavor_text" entry from the "pokemon-species" response in the details that you store. They are in the "flavor_text_entries" key. The text includes special characters such a line breaks ("\n"), form feeds ("\f") and soft hyphens ("\xad") that will need to be stripped out make the text look nice when shown.
   - *Your "`extract_english()`" function will be useful to obtain the English text.*
   - *Include the text when viewing Pokémon details in this program and the GUI program.*

3. When adding a Pokémon, include their base stats in the details that you store. The stats are HP, Attack, Defense, Special-Attack, Special-Defense, and Speed and are contained in a list in the "stats" key of the "Pokémon" response. It is up to you how you obtain/structure them.
   - *Include the stats when viewing Pokémon details in this program and the GUI program.*

4. Make it so that the search feature also searches within a Pokémon's types. Since types are stored as a list of strings, an "in" comparison will check if the search term exactly matches one of the strings in the list, i.e. No partial matches of type names. This is fine.

5. Let users use the "add", "search", "view" and "delete" options more efficiently by allowing input of "a <name>", "s <search term>", "v <number>" and "d <number>". For example, instead of needing to type "a" and then "ekans", the user could type "a ekans", and instead of needing to type "v" then "2", the user could type "v 2".
   - *This feature takes significant planning and effort to implement efficiently.*

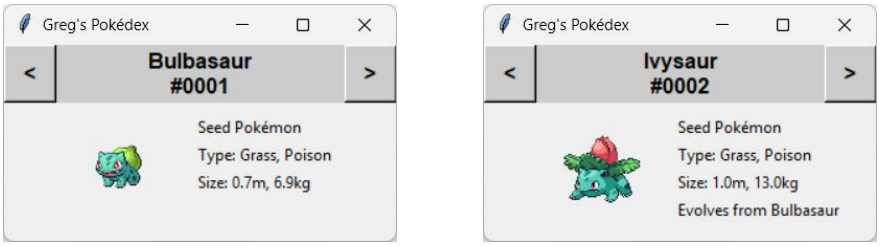


Overview of "pokedex.py"

"pokedex.py" is a program with a Graphical User Interface (GUI), covered in Module 9. It should be coded in an Object Oriented style, covered in Module 8. It can be implemented in under 125 lines of code (although optional additions may result in a longer program). *This is not a limit or a goal, but you should ask your tutor for advice if your program significantly exceeds it.* Everything you need to know for this program is covered in the first 9 modules of the unit.

You must use the "`tkinter`" and the "`tkinter.messagebox`" modules to create the GUI.

This program uses the data from the "pokemon.txt" file. Like the admin program, this program should load data from the file *once only* – when the program begins. The program shows the details of the first Pokémon in the list, along with buttons to navigate between other Pokémon:



The exact layout and appearance of the program is up to you, but it must include the image of the Pokémon and all the data in its dictionary. Format/present the number, type and size as pictured.

The Pokémon's name and number should be emphasised (e.g. big/bold text, or a different colour).

The following pages detail how to implement the program.

Constructor of the GUI Class of "pokedex.py"

The **constructor** (the "`__init__()`" method) of your GUI class must implement the following:

1. Try to open the "pokemon.txt" file in read mode and load the JSON data from the file into an attribute named "`self.data`", and then close the file.
   - *If any exceptions occur (due to the file not existing, or it not containing valid JSON data), show an error messagebox with a suitable message. Include a "`return`" statement in the exception handler to immediately halt the constructor so that the program ends cleanly.*
   - *After loading the data, check whether the `self.data` list is empty. If so, show an error messagebox with a suitable message and end the program in the same way as described above.*

2. Create the main window of the program and give it a title involving your name and "Pokédex" (e.g. "Greg's Pokédex"), and create a "`self.index`" attribute (set it to 0) to keep track of the current Pokémon within `self.data`.
   - *You are welcome to set other main window settings to make the program look/behave as desired.*
   - *If you give the window a custom icon, be sure to include it in your Git bundle.*

3. Use `Frame`, `Label` and `Button` widgets from the "`tkinter`" module to create all the GUI elements needed to display the image and details of a Pokémon (see Page 10).
   - *Save time by mocking up the GUI to figure out the widgets you need before you start writing the code.*
   - *See Reading 9.1 for information regarding various settings that can be applied to widgets to make them appear with the desired padding, colour, font, size, etc.*
   - *A `PhotoImage` widget will be needed to display the image – see Task 3 of Workshop 9.*
   - *You can specify placeholder text for the labels while working on the layout, but the text of the labels will be set/changed via the "`show_pokemon()`" method once it is implemented.*

4. Lastly, call the "`show_pokemon()`" method to display the details of the current/first Pokémon in the GUI, and then call "`tkinter.mainloop()`" to start the main loop.

That is all that the constructor requires. The following pages detail the methods needed in the class, and some optional additions. You are *not* required to submit pseudocode for "pokedex.py".

Note: GUIs and OOP are massive topics that we do not explore in depth in the unit. As such, this program will involve research, experimentation and adapting of examples. Your ability to create a program despite having limited familiarity in these topics is part of what is being tested.

Do not be alarmed or concerned if you find this program difficult for those reasons, and be sure to check the discussion board regularly for tips and examples.

Methods in the GUI class of "pokedex.py"

This program requires you to define 3 methods as part of the GUI class: "`show_pokemon()`", "`previous()`" and "`next()`".

1. The "`show_pokemon()`" method is called in the constructor, and by the `previous()` and `next()` methods. It displays the image and details of the current Pokémon in the GUI. This involves using the details in the current Pokémon's dictionary (i.e. whichever item of `self.data` is being pointed at by `self.index`) to assign strings to the "text" of the `Label` widgets you created in the constructor.
   - *Changing the image involves setting the "file" of the `PhotoImage` widget.*
   - *Only show the "Evolves from..." line if the "evolves_from" key exists in the Pokémon's dictionary (setting the text to an empty string if the key doesn't exist is fine – no need to destroy/unpack).*
   - *Reading 9.1 covers how to configure settings of widgets.*

2. The "`previous()`" method is called when the user clicks the "Previous"/"<" button. It should subtract 1 from `self.index` and then call the `show_pokemon()` method. If `self.index` is already at 0, it should do nothing.

3. The "`next()`" method is called when the user clicks the "Next"/">" button. It should add 1 to `self.index` and then call the `show_pokemon()` method. If `self.index` is already at the last item in the `self.data` list, it should do nothing.

These methods are all that are required to implement the functionality of the program, but you may write/use additional methods if you feel they improve your program.



Optional Additions and Enhancements for "pokedex.py"

Below are suggestions for minor additions and enhancements that you can implement to further test and demonstrate your programming ability.

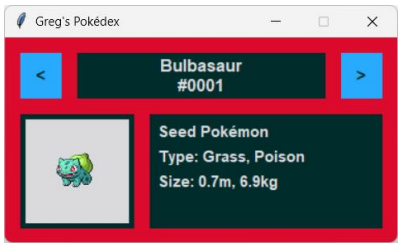
They are *not required,* and you can earn full marks without implementing them.

1. Disable the "Previous"/"<" button while `self.index` is at 0, and disable the "Next"/">" button while `self.index` is at the last item in the `self.data` list.

2. Let the user navigate between Pokémon using the left and right arrow keys on the keyboard.
   - *This involves using the "`bind()`" method on the main window.*

3. When adding a Pokémon in "admin.py", include the Pokémon's colour in the details that you store. It is in the "name" key of the "color" dictionary of the "pokemon-species" response. Use the Pokémon's colour somewhere in the GUI, e.g. as a border around the image.
   - *The "highlightthickness" of a `Label` sets border width, and "highlightbackground" sets its colour. All of the Pokémon colour names are recognised in as values for colours in `tkinter`.*

4. Make your GUI better resemble a Pokédex in colours/layout, or simply make it look fancier, e.g.



5. Make it possible for the user to view a larger image of the Pokémon by clicking on the image. Whether this is shown in the main window or a separate window is up to you.
   - *`PhotoImage` objects have a "`zoom()`" method that can return a magnified version of the image.*

Referencing Out-of-Unit Code

If your program contains any built-in functions or programming techniques *not* covered in the Core Content of a module from this unit, you must reference them within your code. This simply requires a comment above the relevant line(s) of code containing a link to the specific webpage where you learned of the function/technique. If your source of knowledge is not a website (e.g. a reading, textbook, or prior learning), reference or explain it in a comment as clearly as possible.

Failure to reference out-of-unit functions or programming techniques will result in *losing marks* in the Code Quality section of the marking key/rubric. If you are not sure whether something was covered in the Core Content of a module, it is recommended that you reference it.

Submission of Deliverables

You should seek feedback on your work from your tutor at least once prior to submitting it.

When ready to submit, **bundle your Git repository** and submit it to the appropriate location in the Assignments area of the unit site. The bundle must contain your **pseudocode for "admin.py"** (PDF format), **source code for "admin.py" and "pokedex.py"** (".py" files), and **any other supporting files** (such as the folder of Pokémon image files). The repository's commit history should reflect the evolution of your work over time. See the "**Using Git for Assessments**" document provided with this brief for details. Your work *will only be marked* if you submit a bundled Git repository.

Include your name and student number at the top of code/pseudocode files.

Academic Integrity and Misconduct

The **entirety of your assignment must be your own work** (unless otherwise referenced) and produced for the current instance of the unit. Any unreferenced content you did not create is academic misconduct (plagiarism). Assignments are submitted to plagiarism checking software which includes previous assignments, and the work submitted by all students in the unit. Submissions are also manually compared and checked. Do not make your work available online, during or after the unit.

Remember that this is an **individual** assignment. *Never give anyone any part of your assignment – even after the due date or after results have been released. Do not work together with other students on individual assignments – you can help someone by explaining a concept or directing them to the relevant resources, but doing any part of the assignment for them or alongside them, or showing them your work, is inappropriate.* An unacceptable level of cooperation between students on an assignment is academic misconduct (collusion). *Using AI tools (e.g., ChatGPT) is also inappropriate for assignment work. If you are having trouble with your assignment, please talk to/email your tutor.* If you are uncertain about plagiarism, collusion or referencing, ask unit staff.

You may be asked to **explain and demonstrate your understanding** of the work you submitted. Your submission should accurately reflect *your* understanding and ability to apply the unit content.

Marking Key

Criteria	Marks
Pseudocode Marks are awarded for submitting pseudocode which suitably represents the design of your admin program. Pseudocode will be assessed based on whether it clearly describes the steps of the program in English, and whether the program is well structured.	**6**
Functionality Marks are awarded for submitting source code that implements the requirements specified in this brief, in Python 3. Code which is not functional or contains syntax errors will lose marks, as will failing to implement requirements/functions as specified.	**16** *Admin Program: 10* *GUI Program: 6*
Code Quality Marks are awarded for submitting well-written source code that is efficient, well-formatted and demonstrates good understanding of the concepts involved. This includes appropriate commenting, referencing external sources, and adhering to best practise.	**10** *Admin Program: 6* *GUI Program: 4*
Version Control Marks are awarded for submitting a bundled Git repository with a commit history that reflects the evolution to your work over time. This includes committing changes at appropriate intervals, and having well-written commit messages.	**3**

Total:	**35**

See the **"Rubric and Marking Criteria"** document provided with this brief for details.

If the authorship of your work is in question, your mark may be adjusted following a live viva.